

ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	Edge DM- Edge AI Tabletop Game Master
Team Name	SomerSault
Registration / Team ID	APC-2026-TN-91865
Contest Track	Gaming, Robotics & Interactive AI
Institution & City	SRMIST, Chennai

Team Members (1–4)

Name	Email	Role
Team Leader	shivani.sarangi16050@gmail.com	
Member 2 (optional)	suppriya.s06@gmail.com	
Member 3 (optional)		
Member 4 (optional)		

1. Project Overview

Problem Statement

What real-world problem does your project solve? Who faces it? 2–5 sentences.

Traditional Tabletop Roleplaying Games (TTRPGs) generally rely on a human Game Master who has knowledge of several rulebooks and worldbuilding experience to be able to run the game for other players, creating a steep barrier of entry that leaves most potential players unable to play. Current digital solutions primarily rely on cloud-based LLMs that cannot be run locally, or interact with physical pieces, failing to give the traditional tabletop experience. **Edge-DM** solves this by acting as a locally run, edge-AI Game Master that can process natural voice input, track physical board pieces, and narrate in real time. This makes sure that tabletop game enthusiasts, casual gamers, and game shops can play fully immersive, plug-and-play game campaigns without needing a human DM or an internet connection.

How Your Project Works

Briefly explain your project and what it does. 100–200 words.

Edge-DM is an offline, Physical AI Dungeon Master running 100% locally on the Arduino UNO Q, utilizing the dual brain architecture of the Arduino UNO Q by merging AI gameplay and narration, computer vision and structured memory on its linux core.

1. **Local Setup and Web Interface:** At boot, the system narrates an opening world prompt over a speaker. Players connect access a web app hosted on the Qualcomm MPU through local WiFi to upload character sheets and set turn order.
2. **Real Time Voice Pipeline:** Players speak into a USB microphone which is put through Whisper STT on the MPU, passes context to a local Llama 3.2 (1B) model, and synthesizes audio narration using Vosk TTS.
3. **Three Tier RAG Memory Engine:** the 32GB eMMC uses SQLite vector search to manage three layers of memory: **The Vault** – Permanent memory with rulebooks, **The Ledger** – complete dialogue logs, and **Short-Term Memory** – contains last 3 turns to maintain story context.
4. **System Prompts:** To maintain memory retention and rule adherence, the code contains multiple game relevant events with system prompts to enforce certain behavior and refer to permanent vault for instructions or information through the ledger. Each event has 15-20 words that triggers the prompts.
5. **Computer Vision Combat:** During combat, a top down OpenCV camera scans ArUco tagged game pieces, and calculates relative distances, attack ranges, and spell ranges.

Why Arduino UNO Q?

How does its on-device AI capability enable your solution?

The **Arduino UNO Q** is the only single board computer that provides the **dual brain architecture** we exactly required to run an offline, autonomous local AI Dungeon Master in real time.

- **Linux core with Qualcomm MPU:** Its 4GB RAM and 32GB eMMC storage natively host our full local AI stack namely, Whisper STT, Vosk TTS, Llama 3.2 (1B), and an offline SQLite vec RAG memory database and python scripts helping us eliminate cloud latency, API costs, and internet dependency.
 - **Real Time Linux Edge Vision:** OpenCV image processing is able to run top down directly on the Linux MPU to track ArUco-tagged miniatures, which helps the system calculate piece movement, spell ranges, and line of sight without needing an external vision co processor.
-

2. Components Used (BOM)

List every component. The Arduino UNO Q purchase proof must be uploaded separately with your submission.

Component	Qty
Arduino UNO Q (ABX00173)	1
5V/3A USB-C Power Adapter Cor	1
USB Web Camera	1
External Speaker	1
External Mic	1

3. System Architecture & Circuit

Step-by-Step Workflow

Example: Sensor collects data → UNO Q runs AI model on-device → Output device acts.

Step-by-Step Workflow

- Step 1: Local Setup and Voice Capture
 - Local Configuration: Upon boot, users get a greeting message and the Qualcomm MPU hosts a local web server over Wi-Fi. Players access the web app on their browsers to set character sheets and turn orders without external cloud dependencies.
 - Voice Capture: Audio streams captured from an external mic run through a local Whisper Speech-To-Text engine directly on the MPU.
- Step 2: Game Engine
 - Speech-to-Text (STT): The MPU receives the raw audio stream and runs local Whisper STT to transcribe spoken player actions into text in real time.
 - System Prompts and Retrieval Augmented Generation: The transcribed action is evaluated by a local Llama 3.2 (1B) LLM that is given to a thin flask server with no game logic merely six endpoints to warm up the language model into memory so that the user doesn't pay the model loading cost.
 - This flask server gives the data to the main python file in which, an optimized Regex Intent Matrix runs and has 300+ trigger words that can be said by the players which are narrowed down to intents such as intent to begin combat, talk to an npc that trigger a system prompt relevant to this intent.
 - The system prompts instruct to a) give a response relevant to the intent (such as commencing npc interaction) b) or check the Permanent vault for rules on how to behave (such as commencing skill checks if the player wants to do an action) and then narrate

words accordingly c) or look through the ledger to retrieve relevant information that the player asks for.

- A 3-tier database on the eMMC is used for performing these prompts correctly using RAG (Retrieval Augmented Generation):

The Vault- is permanent memory consisting of the ttrpg rulebook and dungeon mastering guide,

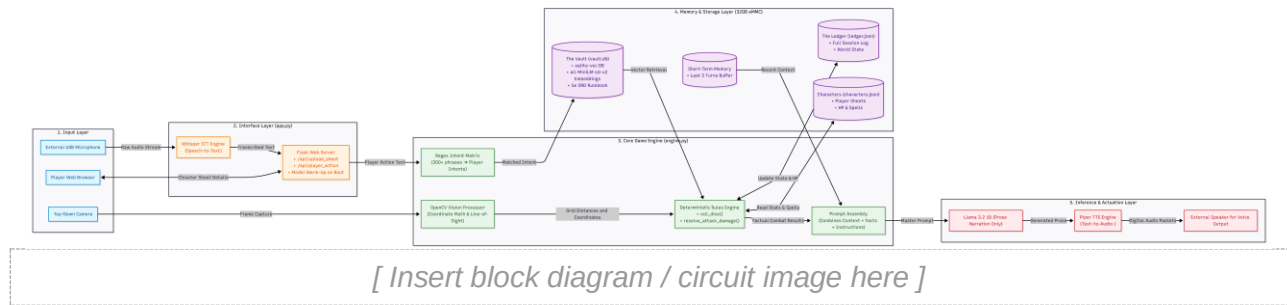
The Ledger – stores the entire log of the game whenever a trigger word was said in game, and

Short-Term Memory - Memory of the last 3 turns to continually maintain relevant context in gameplay. These memories are parsed through using sqlite vector search to prevent latency.

- The system also continually refers to the character sheets received from the webapp continually during turns as players update game relevant details like prepared spells to behave according to the updated character information. As the game progresses, data such as remaining HealthPoints of characters, if the characters are on death saves and previous npc interactions are deposited on the webapp without manual insertion.
- Computer Vision Spatial Math: Once combat mode is triggered, The Camera with OpenCV scans ArUco-tagged miniatures on the board, gives the data to the MPU. As a 1b model is inefficient in calculating combat math, we have opted to run it with python and the script runs the combat math that converts physical grid positions into coordinate data to calculate spell ranges, attack distances, and line-of-sight mechanics. This data is given to the AI to then narrate the happenings in game that occur due to the damages. This ensures smooth and autonomous gameplay for maximum immersion as the players don't have to spend time calculating the damage numbers.
- Narrative & Command Generation: Llama 3.2 synthesizes narrative text according to all the parameters described previously and feeds the text directly into Vosk TTS for real-time speech generation.
- Step 3: Audio Narration
- Audio Playback: The MPU streams synthesized digital audio packets and outputs sound on the external speaker for voice narration and sound effects without audio stutter.

Block Diagram & Circuit Schematic

Paste your block diagram and circuit/wiring diagram below. Hand-drawn photos, Fritzing, KiCad — all accepted. Label pin connections.



4. AI / ML Model Details

If your project does not use AI/ML, write “Not Applicable” — but note that Physical AI projects score higher on Innovation.

Model Used	Llama 3.2 1B
Training Platform	NA. (with reason)
Accuracy	NA, prose cannot be tested for accuracy.
Dataset	Dungeons and Dragons System Reference Document, Sly Flourish's Lazy GM Resource Document.

Brief Description & Limitations

How does the model work in your project? When does it fail or degrade?

- For a 1-billion parameter model to behave as a storytelling game master without hallucinating math or memory about previous events, we used Retrieval Augmented Generation (RAG) through python scripts to give all the necessary information such as dice roll math and previous event information.
- A fast regex matrix classifies the intent (300+ phrases into 12 intents), while the exact rules are retrieved from the SQLite vector vault. Python's deterministic engine handles dice rolls, hit points, and ArUco combat ranges. The calculated results are injected into Llama 3.2 1B to synthesize natural narrative speech which is output via Vosk TTS on a speaker.

5. Code Structure

Briefly explain the main functions. Example: `setup()` → initialises sensors; `loop()` → main execution; `detectObject()` → detection logic.

Code structure

The system is three Python files with a strict separation of concerns: one builds the knowledge base offline, one holds the game logic, one exposes it over HTTP.

`build_vault.py` — offline preparation (run once)

Converts the rulebook into something searchable by meaning rather than keywords. It reads the D&D 5e SRD and Lazy DM's Guide JSON files, passes each rule through a local embedding model (all-MiniLM-L6-v2) to produce a 384-dimension vector, and writes both the vector and the original text into `vault.db`, a SQLite database using the `sqlite-vec` extension. Two linked tables: `vault_embeddings` holds the maths, `vault_text` holds the readable rule, joined on `rowid`. This runs once during setup, not during play, so the cost never appears in gameplay latency.

`engine.py` — the game brain

Holds all reasoning and state. Its main responsibilities:

Intent detection — `INTENT_MATRIX` maps roughly 300 trigger phrases to twelve intents (combat, horde, NPCs, loot, skill checks, exploration, rest, memory recall, session end). Compiled to regex at startup so classification is effectively instant. Rulebook retrieval — `get_vault_context()` embeds the player's sentence and finds the nearest rules in `vault.db` by vector distance. Measured at about 0.2 seconds. Deterministic resolution — `roll_dice()` and `resolve_attack_damage()` roll real dice in Python and apply damage to the target's hit points. Damage dice come from the caster's own character sheet first, the rulebook second. Prompt assembly — `build_master_prompt()` combines the character sheet, retrieved rules, recent history, and an intent-specific instruction into a single prompt. Persistent memory — `ledger.json` stores the full session transcript and world state; `characters.json` stores every character sheet. Both survive restarts. Narration — `generate_narration()` sends the prompt to a local Llama 3.2 1B model through Ollama on 127.0.0.1. No network request ever leaves the board.

`app.py` — the interface layer

A thin Flask server, deliberately containing no game logic. It serves the web interface and exposes six endpoints: `/api/upload_sheet` and `/api/update_stat` for character data, `/api/session_status` and `/api/start_session` for session setup, `/api/player_action` for each turn, and `/api/update_ledger` for world-state changes. It also warms the language model into memory at startup so the first player turn doesn't pay the model-loading cost.

One turn, end to end

A player says "I cast Ray of Frost at the goblin scout." Flask receives it and hands it to the engine. The intent matrix classifies it as combat. The vault returns the relevant rules by vector similarity. Python looks up Ray of Frost on the character sheet, rolls 1d8, subtracts the result from the goblin's hit points, and saves the new state. Only then is a prompt built — one that tells the model what happened and asks it to describe it. The response is spoken aloud, and the exact numbers are returned separately in a `combat_result` field.

The central design decision

A 1-billion-parameter model cannot be trusted with arithmetic since in early testing it invented enemy counter-attacks and produced incorrect hit point totals. Rather than fight this with prompt

engineering, we removed the responsibility entirely. Python owns every number; the model owns only prose. The numbers shown to players come from `combat_result`, never parsed from the model's text. This is what makes a small offline model viable as a rules referee: it is never asked to be one.

GitHub Repository	https://github.com/...
Demo Video Link	YouTube / Drive — public access

6. Testing & Results

How did you test the complete system? What were the results? Include accuracy, response time, edge cases.

Methodology & Testing Setup

- **Memory & Text Retrieval (SQLite FTS5 & Vector DB):** Evaluated context retrieval speed and relevance by executing natural language dialogue queries (e.g., asking to recall past NPC dialogue or specific D&D 5e SRD spell rules).
- **Board Vision & Combat Processing:** Tested board scanning using an overhead camera tracking 4x4 ArUco markers on an 8x8 grid. Tested spatial Euclidean distance calculations and automated HP/dice resolution via Python.
- **System End-to-End Test:** Ran turn-based gameplay loops connecting the Flask API, intent detection engine, vector/FTS database, and local Llama 3.2 1B model via Ollama.

Edge Cases & Resolutions

- **Edge Case: Arithmetic Hallucinations in LLM**
 - *Issue:* The 1B parameter model hallucinated incorrect HP totals and fabricated enemy counter-attacks during initial tests.
 - *Resolution:* Offloaded all mathematical and combat calculations entirely to Python (`engine.py`); the LLM was restricted strictly to generating descriptive prose based on structured JSON data.

Project Images (2–3)

[Insert 2–3 photos of your project working]

7. Challenges, Learnings & Future Improvements

Challenges Faced

e.g. sensor calibration, model optimisation, power management.

- Challenges Faced
- LLM Arithmetic Hallucinations & Math Reliability: When we tested it initially, we found out that Llama 3.2 1B frequently made errors on dice calculations, inverted hit point logic, and hallucinated the damage numbers. The combat mode performed no differently than out of combat narration/adventuring.
- To fix this, We stripped all rule and damage calculations from the LLM. Python (engine.py) handles all of the game math deterministically, passing raw facts to the LLM so that it could do the narration and storytelling seamlessly.
- On-Device Memory & Buffer Overhead: We figured that running continuous OpenCV camera streams alongside our local LLM models risked causing Out-Of-Memory (OOM) issues. Mitigation: We made sure the camera only runs when the players are in combat mode.

First-Turn Latency Overhead: Cold-starting the LLM caused delays on the first turn. So, we integrated a thin flask server deliberately containing no game logic (app.py). It serves as the web interface and has six endpoints. This warms up the LLM into memory at startup so that players don't have to pay the model loading cost.

What You Learned & What's Next

Summarise what you built, what you learned, and 2–3 future improvements.

- We built Edge-DM, an offline physical AI Dungeon Master that runs storytelling Table Top Role Playing Games, 100% locally on the Arduino UNO Q.
- The board's dual-brain architecture, helps us combine the AI local generative prose (Llama 3.2 1B) and use python scripts on its 32 GB eMMC memory to run a RAG system that performs seamlessly as a game master without hallucinating. The board's powerful 4gb MPU also helped us integrate a speech to text(Whisper) and text to speech(Piper) pipeline directly on the MPU unit to deliver immersive gameplay. We also built automated combat calculation through arUco tag detection through OpenCV

computer vision along. A local webapp is hosted on the board's local WiFi to provide character sheet details for the narrative, it has two way communication as the system automatically updates details such as Hit Points on the webapp without manual intervention.

- What We Learned:
- We learned that small 1-billion-parameter edge models cannot be trusted with arithmetic or complex rule enforcement and so, the key to making physical AI reliable is strict separation of concerns. Python handles all of state tracking, dice rolls, and coordinate math, leaving the LLM to focus purely on generating narrative, evolving prose.
- Regex Pre-Filtering for RAG: Running vector search across massive databases introduced latency overhead. So, we pre filtered user input with a fast, regex compiled INTENT MATRIX of 300+ phrases mapped to 12 intents which decreases the RAG retrieval time massively.
- Resource Constraints on Edge Hardware: Running continuous camera streams alongside LLMs can easily trigger Out-Of-Memory (OOM) crashes, therefore we limited the camera runtime to remain within combat mode.

Future Improvements

Expand the STM32 MCU's RPC bridge to dynamically control ambient room lighting and localized audio effects tied to live game states (e.g., flashing red lights during combat rounds).

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	23/8/26
------	---------